

CLONE CODING PROJECT

당근마켓 클론

종합 기획안

Flutter 3.32 · Riverpod 2.6.1 · GoRouter 14.x

Supabase · Firebase · Naver Map

패키지명 com.nomad.karrot

아키텍처 Clean Architecture + Riverpod

기반 프로젝트 com.nomad.baemin 구조 이식

개발 도구 Claude Code + Android Studio MCP

제작 노마드수익화연구소 · 디지털노마드연구소

CLAUDE.md 포함
AI 바이브코딩 즉시 적용

중고거래마켓 클론 — 종합 기획안 (CLAUDE.md)

> `com.nomad.karrot` | Flutter 3.32 / Dart 3.8 | Riverpod 2.6.1 | GoRouter 14.x | Supabase 2.8

---

1. 프로젝트 개요

항목	내용
앱 이름	당근클론 (Karrot Clone)
패키지명	`com.nomad.karrot`
레퍼런스 앱	중고거래마켓 (Karrot Market)
개발 방식	Claude Code + Android Studio (JetBrains MCP)
기반 아키텍처	동일 아키텍처 기반, 도메인 레이어만 교체
minSdk	26 (Android 8.0)
targetSdk	35 (Android 15)

핵심 가치 제안

> "GPS 기반 동네 인증 → 중고 상품 등록 → 실시간 채팅 → 매너온도 기반 신뢰 거래"

---

2. 기술 스택 (확정)

Flutter / Dart

```toml
# pubspec.yaml 핵심 의존성
flutter: 3.32.x
dart: 3.8.x
```

의존성 전체 목록

패키지	버전	용도
`flutter_riverpod`	2.6.1	전역 상태관리
`riverpod_annotation`	2.6.1	코드 생성
`go_router`	14.x	선언형 라우팅
`supabase_flutter`	2.8.x	Auth + DB + Realtime + Storage
`firebase_core`	3.x	Firebase 초기화
`firebase_auth`	5.x	SMS 전화번호 인증
`firebase_messaging`	15.x	FCM 푸시 알림
`kakao_flutter_sdk_user`	1.9.x	카카오 소셜 로그인
`naver_login_sdk`	3.0.x	네이버 소셜 로그인
`flutter_naver_map`	1.3.x	동네 범위 지도 표시
`geolocator`	13.x	GPS 위치 획득
`geocoding`	3.x	좌표 → 주소 변환
`image_picker`	1.1.x	다중 이미지 선택
`cached_network_image`	3.4.x	이미지 캐싱
`freezed`	2.5.x	불변 모델
`drift`	2.x	로컬 DB (오프라인 캐싱)
`flutter_secure_storage`	9.x	토큰 안전 저장
`permission_handler`	11.x	권한 관리
`dio`	5.x	HTTP 클라이언트 (카카오 주소 API)
`intl`	0.19.x	날짜/가격 포맷
`timeago`	3.7.x	"3분 전" 상대 시간
`build_runner`	2.4.x	코드 생성
`riverpod_generator`	2.6.x	Provider 자동 생성
`freezed_annotation`	2.5.x	Freezed 어노테이션

3. 아키텍처

전체 구조

```
Clean Architecture + Riverpod + GoRouter
──────────────────────────────────────────
Presentation Layer  →  features/{feature}/presentation/
Domain Layer        →  features/{feature}/domain/
Data Layer          →  features/{feature}/data/
Core                →  core/ (DI, Router, Supabase, Theme)
```

인증 전략 (3-Mode)

Mode	방식	우선순위
MODE1	Supabase 이메일 / Google / Apple	보조
MODE2	Firebase SMS 전화번호 인증	**메인** (당근 특성)
MODE3	카카오 / 네이버 → Supabase Edge Function JWT 교환	보조

> 중고거래마켓은 전화번호 인증이 핵심. MODE2를 기본 로그인 탭으로 배치.

---

4. 화면 설계 (Screen Architecture)

BottomNav 5탭 구조

탭 인덱스	아이콘	화면명	라우트
0		홈 (동네 피드)	`/home`
1		동네생활	`/community`
2		글쓰기 (FAB)	`/product/write`
3		채팅	`/chat`
4		나의당근	`/my`

전체 화면 목록

인증 (Auth)

화면	라우트	설명
SplashScreen	`/`	토큰 확인 → 분기
OnboardingScreen	`/onboarding`	동네 서비스 소개 3장
LoginScreen	`/login`	3탭 (전화번호 / 이메일 / 소셜)
PhoneVerifyScreen	`/login/verify`	Firebase SMS OTP 6자리
LocationSetupScreen	`/setup/location`	최초 동네 인증 (필수)

홈 (Home)

화면	라우트	설명
HomeScreen	`/home`	카테고리 필터 + 피드 리스트
SearchScreen	`/home/search`	동네 범위 내 키워드 검색
CategoryFilterSheet	BottomSheet	카테고리 선택 (30종)
LocationRangeSheet	BottomSheet	동네 범위 4단계 설정

상품 (Product)

화면	라우트	설명
ProductDetailScreen	`/product/:id`	상품 상세 (이미지 갤러리 + 지도)
ProductWriteScreen	`/product/write`	등록 (다중 이미지, 카테고리, 가격)
ProductEditScreen	`/product/:id/edit`	수정
PriceNegotiateSheet	BottomSheet	가격 제안
LikeListScreen	`/my/likes`	관심 목록

채팅 (Chat)

화면	라우트	설명
ChatListScreen	`/chat`	채팅방 목록 (최근 메시지 + 미읽음 배지)
ChatRoomScreen	`/chat/:roomId`	실시간 채팅 (Supabase Realtime)
ChatProductInfoBar	채팅방 상단 고정	거래 상품 정보 + 상태 변경

동네 (Location)

화면	라우트	설명
LocationVerifyScreen	`/location/verify`	GPS 인증 or 직접 검색
LocationSelectScreen	`/location/select`	동네 목록 관리 (최대 2개)
LocationMapScreen	`/location/map`	네이버 지도 + 동네 범위 시각화

동네생활 (Community)

화면	라우트	설명
CommunityFeedScreen	`/community`	동네 게시글 피드
PostDetailScreen	`/community/:id`	게시글 상세 + 댓글
PostWriteScreen	`/community/write`	글쓰기

나의당근 (My)

화면	라우트	설명
MyScreen	`/my`	프로필 + 매너온도 + 메뉴
SellerProfileScreen	`/profile/:userId`	타인 프로필 (판매 목록 + 후기)
MySalesScreen	`/my/sales`	내 판매 내역 (상태별 필터)
MyPurchaseScreen	`/my/purchases`	내 구매 내역
MannerReviewScreen	`/review/write`	거래 완료 후 매너 평가
SettingsScreen	`/settings`	알림, 계정, 로그아웃

5. Supabase 데이터베이스 스키마

5-1. 사용자 프로필

```SQL
CREATE TABLE user_profiles (
  id UUID PRIMARY KEY REFERENCES auth.users ON DELETE CASCADE,
  nickname TEXT NOT NULL,
  profile_image_url TEXT,
  phone_number TEXT UNIQUE,
  temperature DECIMAL(4,1) DEFAULT 36.5,  -- 매너온도
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;
CREATE POLICY "본인만 수정" ON user_profiles FOR UPDATE USING (auth.uid() = id);
CREATE POLICY "전체 조회 가능" ON user_profiles FOR SELECT USING (true);
```

5-2. 동네 인증

```sql
CREATE TABLE user_locations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users NOT NULL,
  region_name TEXT NOT NULL,        -- "서울특별시 마포구 망원동"
  dong_name TEXT NOT NULL,          -- "망원동" (표시용)
  lat DOUBLE PRECISION NOT NULL,
  lng DOUBLE PRECISION NOT NULL,
  range_km INTEGER DEFAULT 2,       -- 0.5 / 2 / 6 / 10 km
  is_primary BOOLEAN DEFAULT true,
  is_verified BOOLEAN DEFAULT false,
  verified_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  CONSTRAINT max_two_locations CHECK (
    (SELECT COUNT(*) FROM user_locations WHERE user_id = user_locations.user_id) <= 2
  )
);
```

5-3. 상품

```sql
CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  seller_id UUID REFERENCES auth.users NOT NULL,
  title TEXT NOT NULL,
  description TEXT,
  price INTEGER,                    -- NULL이면 무료나눔
  is_price_negotiable BOOLEAN DEFAULT false,
  is_free BOOLEAN DEFAULT false,
  category TEXT NOT NULL,
  status TEXT DEFAULT 'selling',    -- selling / reserved / sold / hidden
  image_urls TEXT[] DEFAULT '{}',   -- Supabase Storage URL 배열
  location_dong TEXT NOT NULL,      -- 표시용 동네명
  lat DOUBLE PRECISION,
  lng DOUBLE PRECISION,
  view_count INTEGER DEFAULT 0,
  like_count INTEGER DEFAULT 0,
  chat_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_products_seller ON products(seller_id);
CREATE INDEX idx_products_status ON products(status);
CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_products_location ON products USING GIST(
  ll_to_earth(lat, lng)
);  -- 거리 기반 필터링

-- RLS
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
CREATE POLICY "판매자만 수정" ON products FOR UPDATE USING (auth.uid() = seller_id);
CREATE POLICY "전체 조회" ON products FOR SELECT USING (status != 'hidden');
CREATE POLICY "본인 등록" ON products FOR INSERT WITH CHECK (auth.uid() = seller_id);
```

5-4. 관심 (Like)

```sql
CREATE TABLE product_likes (
  user_id UUID REFERENCES auth.users NOT NULL,
  product_id UUID REFERENCES products ON DELETE CASCADE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, product_id)
);
```

5-5. 채팅

```sql
CREATE TABLE chat_rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id UUID REFERENCES products ON DELETE SET NULL,
  buyer_id UUID REFERENCES auth.users NOT NULL,
  seller_id UUID REFERENCES auth.users NOT NULL,
  last_message TEXT,
  last_message_at TIMESTAMPTZ,
  buyer_unread_count INTEGER DEFAULT 0,
  seller_unread_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(product_id, buyer_id)       -- 상품당 1개 채팅방
);

CREATE TABLE chat_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID REFERENCES chat_rooms ON DELETE CASCADE NOT NULL,
  sender_id UUID REFERENCES auth.users NOT NULL,
  content TEXT,
  image_url TEXT,
  message_type TEXT DEFAULT 'text', -- text / image / system
  is_read BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_messages_room ON chat_messages(room_id, created_at DESC);

-- Realtime 활성화
ALTER PUBLICATION supabase_realtime ADD TABLE chat_messages;
ALTER PUBLICATION supabase_realtime ADD TABLE chat_rooms;
```

5-6. 매너 리뷰

```sql
CREATE TABLE manner_reviews (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reviewer_id UUID REFERENCES auth.users NOT NULL,
  target_id UUID REFERENCES auth.users NOT NULL,
  product_id UUID REFERENCES products,
  review_type TEXT NOT NULL,        -- 'positive' / 'negative'
  score DECIMAL(3,1) NOT NULL,      -- +0.1 ~ +0.5 / -0.1 ~ -0.5
  keywords TEXT[],                  -- 선택형 후기 키워드 배열
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(reviewer_id, product_id)   -- 상품당 1회만
);

-- 매너온도 자동 업데이트 트리거
CREATE OR REPLACE FUNCTION update_temperature()
RETURNS TRIGGER AS $$
BEGIN
  UPDATE user_profiles
  SET temperature = GREATEST(0, LEAST(99.9, temperature + NEW.score))
  WHERE id = NEW.target_id;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_temperature
AFTER INSERT ON manner_reviews
FOR EACH ROW EXECUTE FUNCTION update_temperature();
```

5-7. 동네생활 게시글

```sql
CREATE TABLE community_posts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  author_id UUID REFERENCES auth.users NOT NULL,
  category TEXT NOT NULL,           -- '동네질문' / '생활정보' / '분실분실' / '동네사건사고' 등
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  image_urls TEXT[] DEFAULT '{}',
  location_dong TEXT NOT NULL,
  view_count INTEGER DEFAULT 0,
  like_count INTEGER DEFAULT 0,
  comment_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE community_comments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  post_id UUID REFERENCES community_posts ON DELETE CASCADE NOT NULL,
  author_id UUID REFERENCES auth.users NOT NULL,
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

6. Supabase Edge Functions

Function명	트리거	역할
`kakao-token-exchange`	POST	카카오 Access Token → Supabase JWT 발급
`naver-token-exchange`	POST	네이버 Access Token → Supabase JWT 발급
`firebase-token-exchange`	POST	Firebase ID Token → Supabase JWT 발급
`location-verify`	POST	GPS 좌표 → 카카오 주소 API → 법정동 반환
`send-push-notification`	DB Trigger	채팅 메시지 → FCM 푸시 발송
`update-product-counts`	DB Trigger	조회수/관심수/채팅수 비동기 업데이트

7. 프로젝트 파일 트리

```
com.nomad.karrot/
├── android/
│   └── app/src/main/
│       ├── AndroidManifest.xml      # 위치권한, 딥링크, FCM
│       └── kotlin/com/nomad/karrot/
│           └── MainActivity.kt
├── ios/
│   └── Runner/
│       ├── Info.plist               # 위치권한 문구, URL Scheme
│       └── AppDelegate.swift
├── lib/
│   ├── main.dart                    # 앱 진입점, Riverpod ProviderScope
│   ├── app.dart                     # MaterialApp.router + GoRouter
│   │
│   ├── core/
│   │   ├── constants/
│   │   │   ├── app_colors.dart      # 당근 컬러 시스템
│   │   │   ├── app_text_styles.dart
│   │   │   └── supabase_constants.dart
│   │   ├── di/
│   │   │   ├── supabase_module.dart # Supabase 초기화
│   │   │   └── firebase_module.dart # Firebase 초기화
│   │   ├── router/
│   │   │   ├── app_router.dart      # GoRouter 전체 라우트 정의
│   │   │   └── route_names.dart     # 라우트 상수
│   │   ├── theme/
│   │   │   └── app_theme.dart       # MaterialTheme (당근 스타일)
│   │   └── utils/
│   │       ├── price_formatter.dart # 가격 포맷 (₩35,000)
│   │       ├── distance_calculator.dart
│   │       └── image_uploader.dart  # Supabase Storage 업로드 유틸
│   │
│   ├── features/
│   │   │
│   │   ├── auth/
│   │   │   ├── domain/
│   │   │   │   ├── models/
│   │   │   │   │   ├── user_session.dart        # Freezed
│   │   │   │   │   └── auth_state.dart          # Freezed
│   │   │   │   ├── repositories/
│   │   │   │   │   └── auth_repository.dart     # abstract
│   │   │   │   └── usecases/
```

```
│   │   │   │       ├── sign_in_with_phone.dart
│   │   │   │       ├── sign_in_with_email.dart
│   │   │   │       ├── sign_in_with_kakao.dart
│   │   │   │       ├── sign_in_with_naver.dart
│   │   │   │       ├── sign_in_with_google.dart
│   │   │   │       ├── verify_sms_code.dart
│   │   │   │       ├── sign_out.dart
│   │   │   │       ├── get_current_user.dart
│   │   │   │       └── watch_auth_state.dart
│   │   │   ├── data/
│   │   │   │   ├── auth_repository_impl.dart
│   │   │   │   └── auth_data_store.dart         # flutter_secure_storage
│   │   │   └── presentation/
│   │   │       ├── screens/
│   │   │       │   ├── splash_screen.dart
│   │   │       │   ├── onboarding_screen.dart
│   │   │       │   ├── login_screen.dart        # 3탭 TabBar
│   │   │       │   └── phone_verify_screen.dart
│   │   │       └── providers/
│   │   │           ├── auth_provider.dart
│   │   │           └── login_provider.dart
│   │   │
│   │   ├── location/
│   │   │   ├── domain/
│   │   │   │   ├── models/
│   │   │   │   │   └── user_location.dart       # Freezed
│   │   │   │   ├── repositories/
│   │   │   │   │   └── location_repository.dart
│   │   │   │   └── usecases/
│   │   │   │       ├── verify_location_by_gps.dart
│   │   │   │       ├── verify_location_by_search.dart
│   │   │   │       ├── get_my_locations.dart
│   │   │   │       ├── add_location.dart
│   │   │   │       ├── delete_location.dart
│   │   │   │       └── set_location_range.dart
│   │   │   ├── data/
│   │   │   │   ├── location_repository_impl.dart
│   │   │   │   └── kakao_address_api.dart       # 카카오 주소 API
│   │   │   └── presentation/
│   │   │       ├── screens/
│   │   │       │   ├── location_setup_screen.dart  # 최초 설정
│   │   │       │   ├── location_verify_screen.dart
│   │   │       │   ├── location_select_screen.dart
```

```
│   │   │       │   └── location_map_screen.dart    # 네이버 지도
│   │   │       └── providers/
│   │   │           └── location_provider.dart
│   │   │
│   │   ├── home/
│   │   │   ├── domain/
│   │   │   │   ├── models/
│   │   │   │   │   └── product_summary.dart     # 피드 카드용 Freezed
│   │   │   │   ├── repositories/
│   │   │   │   │   └── feed_repository.dart
│   │   │   │   └── usecases/
│   │   │   │       ├── get_feed_products.dart   # 동네 범위 필터
│   │   │   │       ├── search_products.dart
│   │   │   │       └── get_categories.dart
│   │   │   ├── data/
│   │   │   │   └── feed_repository_impl.dart
│   │   │   └── presentation/
│   │   │       ├── screens/
│   │   │       │   ├── home_screen.dart
│   │   │       │   └── search_screen.dart
│   │   │       ├── widgets/
│   │   │       │   ├── product_card.dart         # 피드 카드 컴포넌트
│   │   │       │   ├── category_filter_chip.dart
│   │   │       │   ├── location_range_sheet.dart
│   │   │       │   └── home_app_bar.dart
│   │   │       └── providers/
│   │   │           ├── feed_provider.dart
│   │   │           └── search_provider.dart
│   │   │
│   │   ├── product/
│   │   │   ├── domain/
│   │   │   │   ├── models/
│   │   │   │   │   ├── product.dart             # 전체 상품 Freezed
│   │   │   │   │   └── product_category.dart    # enum 30종
│   │   │   │   ├── repositories/
│   │   │   │   │   └── product_repository.dart
│   │   │   │   └── usecases/
│   │   │   │       ├── get_product_detail.dart
│   │   │   │       ├── create_product.dart
│   │   │   │       ├── update_product.dart
│   │   │   │       ├── delete_product.dart
│   │   │   │       ├── update_product_status.dart
│   │   │   │       ├── toggle_like.dart
```

```
│   │   │   │       ├── get_my_sales.dart
│   │   │   │       └── get_liked_products.dart
│   │   │   ├── data/
│   │   │   │   └── product_repository_impl.dart
│   │   │   └── presentation/
│   │   │       ├── screens/
│   │   │       │   ├── product_detail_screen.dart
│   │   │       │   ├── product_write_screen.dart
│   │   │       │   ├── product_edit_screen.dart
│   │   │       │   ├── my_sales_screen.dart
│   │   │       │   └── liked_products_screen.dart
│   │   │       ├── widgets/
│   │   │       │   ├── image_gallery.dart        # 페이지뷰 갤러리
│   │   │       │   ├── multi_image_picker.dart   # 최대 10장
│   │   │       │   ├── price_input_field.dart
│   │   │       │   ├── category_picker.dart
│   │   │       │   ├── product_bottom_bar.dart   # 좋아요+채팅+가격
│   │   │       │   ├── status_badge.dart         # 판매중/예약중/판매완료
│   │   │       │   └── price_negotiate_sheet.dart
│   │   │       └── providers/
│   │   │           ├── product_provider.dart
│   │   │           ├── product_write_provider.dart
│   │   │           └── like_provider.dart
│   │   │
│   │   ├── chat/
│   │   │   ├── domain/
│   │   │   │   ├── models/
│   │   │   │   │   ├── chat_room.dart          # Freezed
│   │   │   │   │   └── chat_message.dart       # Freezed
│   │   │   │   ├── repositories/
│   │   │   │   │   └── chat_repository.dart
│   │   │   │   └── usecases/
│   │   │   │       ├── get_chat_rooms.dart
│   │   │   │       ├── get_or_create_chat_room.dart
│   │   │   │       ├── send_message.dart
│   │   │   │       ├── send_image_message.dart
│   │   │   │       ├── watch_messages.dart     # Realtime Stream
│   │   │   │       ├── mark_as_read.dart
│   │   │   │       └── change_product_status.dart
│   │   │   ├── data/
│   │   │   │   └── chat_repository_impl.dart   # Supabase Realtime
│   │   │   └── presentation/
│   │   │       ├── screens/
```

```
│   │   │       │   ├── chat_list_screen.dart
│   │   │       │   └── chat_room_screen.dart
│   │   │       ├── widgets/
│   │   │       │   ├── message_bubble.dart
│   │   │       │   ├── message_input_bar.dart
│   │   │       │   ├── chat_product_info_bar.dart  # 상단 고정 상품 정보
│   │   │       │   ├── chat_room_tile.dart
│   │   │       │   └── unread_badge.dart
│   │   │       └── providers/
│   │   │           ├── chat_list_provider.dart
│   │   │           └── chat_room_provider.dart
│   │   │
│   │   ├── community/
│   │   │   ├── domain/
│   │   │   │   ├── models/
│   │   │   │   │   ├── community_post.dart      # Freezed
│   │   │   │   │   └── community_comment.dart   # Freezed
│   │   │   │   ├── repositories/
│   │   │   │   │   └── community_repository.dart
│   │   │   │   └── usecases/
│   │   │   │       ├── get_posts.dart
│   │   │   │       ├── get_post_detail.dart
│   │   │   │       ├── create_post.dart
│   │   │   │       ├── create_comment.dart
│   │   │   │       └── toggle_post_like.dart
│   │   │   ├── data/
│   │   │   │   └── community_repository_impl.dart
│   │   │   └── presentation/
│   │   │       ├── screens/
│   │   │       │   ├── community_feed_screen.dart
│   │   │       │   ├── post_detail_screen.dart
│   │   │       │   └── post_write_screen.dart
│   │   │       ├── widgets/
│   │   │       │   ├── post_card.dart
│   │   │       │   └── comment_tile.dart
│   │   │       └── providers/
│   │   │           └── community_provider.dart
│   │   │
│   │   └── my/
│   │       ├── domain/
│   │       │   ├── models/
│   │       │   │   └── manner_review.dart       # Freezed
│   │       │   ├── repositories/
```

```
│   │       │   │   └── profile_repository.dart
│   │       │   └── usecases/
│   │       │       ├── get_my_profile.dart
│   │       │       ├── update_profile.dart
│   │       │       ├── get_seller_profile.dart
│   │       │       ├── write_manner_review.dart
│   │       │       └── get_manner_reviews.dart
│   │       ├── data/
│   │       │   └── profile_repository_impl.dart
│   │       └── presentation/
│   │           ├── screens/
│   │           │   ├── my_screen.dart
│   │           │   ├── seller_profile_screen.dart
│   │           │   ├── manner_review_screen.dart
│   │           │   └── settings_screen.dart
│   │           ├── widgets/
│   │           │   ├── temperature_gauge.dart    # 온도계 UI
│   │           │   ├── manner_keyword_chip.dart
│   │           │   └── profile_header.dart
│   │           └── providers/
│   │               ├── profile_provider.dart
│   │               └── review_provider.dart
│   │
│   └── shared/
│       ├── widgets/
│       │   ├── karrot_app_bar.dart
│       │   ├── karrot_button.dart
│       │   ├── loading_overlay.dart
│       │   ├── error_view.dart
│       │   ├── empty_view.dart
│       │   └── bottom_nav_shell.dart   # ShellRoute 래퍼
│       └── extensions/
│           ├── string_extensions.dart
│           ├── datetime_extensions.dart
│           └── int_extensions.dart      # 가격 포맷
│
├── supabase/
│   ├── migrations/
│   │   ├── 001_user_profiles.sql
│   │   ├── 002_user_locations.sql
│   │   ├── 003_products.sql
│   │   ├── 004_product_likes.sql
│   │   ├── 005_chat_rooms.sql
```

```
│   │   ├── 006_chat_messages.sql
│   │   ├── 007_manner_reviews.sql
│   │   └── 008_community.sql
│   └── functions/
│       ├── kakao-token-exchange/
│       │   └── index.ts
│       ├── naver-token-exchange/
│       │   └── index.ts
│       ├── firebase-token-exchange/
│       │   └── index.ts
│       ├── location-verify/
│       │   └── index.ts
│       └── send-push-notification/
│           └── index.ts
│
├── CLAUDE.md                        # AI 개발 컨텍스트 문서 (본 파일)
└── pubspec.yaml
```

---

8. GoRouter 전체 라우트 정의

DART

```dart
// core/router/app_router.dart
final appRouter = GoRouter(
  initialLocation: '/splash',
  redirect: (context, state) {
    final isLoggedIn = ref.read(authProvider).isAuthenticated;
    final hasLocation = ref.read(locationProvider).hasPrimaryLocation;
    if (!isLoggedIn) return '/login';
    if (!hasLocation) return '/setup/location';
    return null;
  },
  routes: [
    GoRoute(path: '/splash',    builder: (_, __) => const SplashScreen()),
    GoRoute(path: '/onboarding',builder: (_, __) => const OnboardingScreen()),
    GoRoute(path: '/login',     builder: (_, __) => const LoginScreen(),
      routes: [
        GoRoute(path: 'verify', builder: (_, state) => PhoneVerifyScreen(
          verificationId: state.extra as String)),
      ],
    ),
    GoRoute(path: '/setup/location', builder: (_, __) => const LocationSetupScreen()),

    // BottomNav ShellRoute
    StatefulShellRoute.indexedStack(
      builder: (_, __, shell) => BottomNavShell(shell: shell),
      branches: [
        StatefulShellBranch(routes: [
          GoRoute(path: '/home', builder: (_, __) => const HomeScreen(),
            routes: [
              GoRoute(path: 'search', builder: (_, __) => const SearchScreen()),
            ],
          ),
        ]),
        StatefulShellBranch(routes: [
          GoRoute(path: '/community', builder: (_, __) => const CommunityFeedScreen(),
            routes: [
              GoRoute(path: ':id', builder: (_, state) => PostDetailScreen(id: state.pathParameters['id
              GoRoute(path: 'write', builder: (_, __) => const PostWriteScreen()),
            ],
          ),
        ]),
        StatefulShellBranch(routes: [
```

```
          GoRoute(path: '/product', redirect: (_, __) => '/home',
            routes: [
              GoRoute(path: 'write', builder: (_, __) => const ProductWriteScreen()),
              GoRoute(path: ':id', builder: (_, state) => ProductDetailScreen(id: state.pathParameters[
                routes: [
                  GoRoute(path: 'edit', builder: (_, state) => ProductEditScreen(id: state.pathParamete
                ],
              ),
            ],
          ),
        ]),
        StatefulShellBranch(routes: [
          GoRoute(path: '/chat', builder: (_, __) => const ChatListScreen(),
            routes: [
              GoRoute(path: ':roomId', builder: (_, state) => ChatRoomScreen(roomId: state.pathParamete
            ],
          ),
        ]),
        StatefulShellBranch(routes: [
          GoRoute(path: '/my', builder: (_, __) => const MyScreen(),
            routes: [
              GoRoute(path: 'sales',     builder: (_, __) => const MySalesScreen()),
              GoRoute(path: 'purchases', builder: (_, __) => const MyPurchaseScreen()),
              GoRoute(path: 'likes',     builder: (_, __) => const LikedProductsScreen()),
              GoRoute(path: 'settings',  builder: (_, __) => const SettingsScreen()),
            ],
          ),
        ]),
      ],
    ),

    // 독립 라우트
    GoRoute(path: '/location/verify',builder: (_, __) => const LocationVerifyScreen()),
    GoRoute(path: '/location/select', builder: (_, __) => const LocationSelectScreen()),
    GoRoute(path: '/location/map',    builder: (_, __) => const LocationMapScreen()),
    GoRoute(path: '/profile/:userId', builder: (_, state) => SellerProfileScreen(userId: state.pathPara
    GoRoute(path: '/review/write',    builder: (_, state) => MannerReviewScreen(args: state.extra as Ma
  ],
);
```

9. 핵심 컬러 시스템

DART

```dart
// core/constants/app_colors.dart
class AppColors {
  // 중고거래마켓 브랜드 컬러
  static const primary        = Color(0xFFFF6F0F);  // 당근 오렌지
  static const primaryLight   = Color(0xFFFFF0E6);  // 오렌지 연한 배경
  static const primaryDark    = Color(0xFFE65C00);

  // 중성색
  static const background     = Color(0xFFFFFFFF);
  static const surfaceGray    = Color(0xFFF4F4F4);
  static const divider        = Color(0xFFEEEEEE);

  // 텍스트
  static const textPrimary    = Color(0xFF212529);
  static const textSecondary  = Color(0xFF868E96);
  static const textTertiary   = Color(0xFFADB5BD);

  // 매너온도 색상
  static const tempBlue       = Color(0xFF3D95FF);  // 36.4 이하
  static const tempGreen      = Color(0xFF1DCC70);  // 36.5 ~ 42.0
  static const tempOrange     = Color(0xFFFF6F0F);  // 42.1 ~ 60.0
  static const tempRed        = Color(0xFFFF4444);  // 60.1 이상

  // 상태 배지
  static const statusSelling  = Color(0xFF1DCC70);  // 판매중
  static const statusReserved = Color(0xFFFF9500);  // 예약중
  static const statusSold     = Color(0xFF868E96);  // 판매완료료
}
```

10. 개발 로드맵

Phase 1 — 인증 + 동네 기반 (Week 1~2)

> 이전 프로젝트의 AUTH 파일 구조를 `com.nomad.karrot`에 이식

순번	파일	우선순위
1	`pubspec.yaml`	최우선
2	`core/router/app_router.dart`	최우선
3	`core/di/supabase_module.dart`	최우선
4	`features/auth/` (기존 구조 이식)	HIGH
5	`features/location/` 전체	HIGH
6	Supabase Migration 001~002	HIGH
7	Edge Function: `location-verify`	HIGH
8	Edge Function: `kakao/naver-token-exchange`	HIGH

Phase 2 — 상품 CRUD + 피드 (Week 3~4)

순번	파일	우선순위
9	Supabase Migration 003~004	HIGH
10	`features/product/domain/` 전체	HIGH
11	`features/product/data/product_repository_impl.dart`	HIGH
12	`features/home/` 전체 (피드 + 검색)	HIGH
13	`shared/widgets/product_card.dart`	HIGH
14	`features/product/presentation/product_write_screen.dart`	HIGH
15	`features/product/presentation/product_detail_screen.dart`	HIGH

Phase 3 — 실시간 채팅 (Week 5)

순번	파일	우선순위
16	Supabase Migration 005~006	HIGH
17	`features/chat/data/chat_repository_impl.dart`	HIGH
18	`features/chat/presentation/chat_list_screen.dart`	HIGH
19	`features/chat/presentation/chat_room_screen.dart`	HIGH
20	Edge Function: `send-push-notification`	MEDIUM

Phase 4 — 매너온도 + 동네생활 + 마이 (Week 6~7)

순번	파일	우선순위
21	Supabase Migration 007~008	MEDIUM
22	`features/my/` 전체	MEDIUM
23	`features/community/` 전체	MEDIUM
24	`shared/widgets/temperature_gauge.dart`	MEDIUM

Phase 5 — 폴리싱 + 배포 (Week 8)

순번	작업	우선순위
25	FCM 푸시 알림 통합 테스트	MEDIUM
26	오프라인 캐싱 (Drift)	LOW
27	네이버 지도 동네 범위 시각화	LOW
28	Play Store / App Store 제출 준비	LOW

11. 코드 재사용 전략

그대로 이식 (Copy & Adapt)

- `AuthModels.dart` → `UserSession`, `AuthState`
- `AuthRepository.dart` → interface 동일
- `AuthRepositoryImpl.dart` → 패키지명만 변경
- `AuthUseCases.dart` (9개 전부)
- `AuthDataStore.dart`
- `AppModules.dart` (Hilt DI → Riverpod으로 이미 변환됨)
- `LoginScreen.dart` → 탭 순서 변경 (전화번호 우선)
- `LoginViewModel.dart` / `LoginProvider`

수정 필요

- `AppRouter` → BottomNav 5탭 + 채팅 라우트 추가
- `SplashScreen` → location 인증 여부 분기 추가

완전 신규

- `location/` feature 전체
- `chat/` feature 전체
- `community/` feature 전체
- Supabase Edge Functions 5개
- `temperature_gauge.dart` 위젯

---

12. 환경 변수 (.env)

DOTENV

```
# Supabase
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_ANON_KEY=eyJ...

# Firebase (google-services.json / GoogleService-Info.plist로 대체 가능)
FIREBASE_PROJECT_ID=nomad-karrot

# Kakao
KAKAO_NATIVE_APP_KEY=abcd1234
KAKAO_REST_API_KEY=efgh5678

# Naver
NAVER_CLIENT_ID=xxxx
NAVER_CLIENT_SECRET=xxxx

# Kakao Map (주소 변환용)
KAKAO_MAP_REST_KEY=xxxx

# Naver Map (지도 표시용)
NAVER_MAP_CLIENT_ID=xxxx
```

13. CLAUDE.md 개발 지시사항

MARKDOWN

```
## 개발 원칙
- 패키지명: com.nomad.karrot
- 아키텍처: Clean Architecture + Riverpod 2.6.1 + GoRouter 14.x
- 기존 프로젝트(com.nomad.baemin)의 auth 구조를 기반으로 이식
- 모든 모델은 Freezed 사용
- Repository는 abstract → impl 분리 필수
- Provider는 riverpod_generator 코드 생성 방식

## 인증 우선순위
- MODE2 (Firebase SMS) 를 로그인 화면 첫 번째 탭으로 배치
- 첫 로그인 후 반드시 동네 인증(LocationSetupScreen) 거쳐야 홈 진입

## Supabase 규칙
- 모든 테이블 RLS 필수 활성화
- Storage 버킷: products-images (public), chat-images (private)
- Realtime: chat_messages, chat_rooms 테이블만 활성화

## 금지 사항
- BuildContext를 Repository/UseCase 레이어에 전달 금지
- StatefulWidget 남용 금지 → ConsumerWidget 우선
- 직접 Supabase.instance 호출 금지 → DI Provider를 통해서만
```